

UNIVERSITY OF Waterloo



Final Project Report

ECE 417 Image Processing

Adaptive background mixture models for real-time tracking

By Chris Stauffer and W.E.L. Grimson

Under the guidance of

Professor Oleg Michailovich

By	San Basnet, Alex Dufresne-Nappert, Gigi Sae-zheng
Student Number	20953944, 20948586, 20936828
Submission Date	April 23rd, 2026

Abstract

Background subtraction is a fundamental problem in image and video processing, enabling applications such as object tracking, computational photography (e.g., synthetic bokeh), and visual effects in film and graphics. Existing approaches often face trade-offs between computational efficiency and generalizability. Classical methods, such as region- or edge-based segmentation, degrade in performance when foreground and background share similar visual features, while neural network-based approaches are computationally intensive and require extensive training, limiting their suitability for resource-constrained systems and unseen environments.

Temporal modelling provides a more robust alternative by leveraging changes over time to distinguish foreground objects from static background elements. This is particularly relevant in continuous video streams, such as those from security cameras, where storing all captured data is inefficient and unnecessary when the primary interest is in dynamic objects.

In this work, we implement and evaluate an adaptive background modelling approach based on a mixture of Gaussians, as proposed by Stauffer and Grimson. This method incrementally updates a probabilistic model of the scene, enabling real-time classification of pixels as foreground or background under varying conditions.

Introduction and Background

The standard model of adaptive background usually takes the following approaches: they average across many images and create an approximation of a background. This simple approach fails in certain cases, such as periodic movements with a high contrast: wind moving tree leaves, or light sparkling on water.

The Gaussian Mixture-based Background/Foreground Segmentation Algorithm, simply called the Stauffer/Grimson method from this point forward method assumes that each pixel value in the background may change; however, it only changes between a few values. For example, a pixel in a footage of water, or a snowy scene, may change quickly between pure white and dark blue. To account for this, the model saves K Gaussian distributions, which effectively track the history of the pixels [1].

The probability of a pixel value can be modelled with the equation below, but as mentioned in the paper, this method is too computationally expensive due to the Gaussian, therefor an approximation explained later is used.

$$P(\chi_t) = \sum_{i=1}^K \omega_{i,t} * \eta(\chi_t, \mu_{i,t}, \Sigma_{i,t})$$

Where K is the number of gaussians recorded/pixel, $\omega_{i,t}$ is the weight of the gaussian i , at a time t , η is the gaussian probability distribution with the parameters: the input vector $\chi_{i,t}$, the mean $\mu_{i,t}$ and the covariance $\Sigma_{i,t}$. Here, the assumption is that the covariances of the RGB pixels are all the same, which trades off accuracy for much less computational power.

These Gaussians are built for every new frame for each pixel. The new pixel value is compared against the K existing Gaussian distributions in its mixture model. A "match" occurs if the value is within $2.5^2 * \text{Variance}$ distance of a distribution's mean. If this match occurs, this pixel value updates that particular Gaussian. This makes sure that the model is robust to noise and small background changes. If no matches are found, the Gaussian with the least weight associated is replaced by a Gaussian centred around the new pixel value. This makes the model adaptive and generalizable to most scenes. To update the Weight of a Gaussian, the following equation is used:

$$\omega_{k,t} = (1 - \alpha)\omega_{k,t-1} + \alpha M_{k,t}$$

Where α is the learning rate, and $M_{k,t}$ is a binary matching state (1 for match, 0 for non-match). This means that more frequently appearing pixel values are reinforced and infrequent values are forgotten. Next, the Gaussian properties, like the Mean and the Variance, are adjusted as follows:

$$\begin{aligned}\mu_t &= (1 - \rho)\mu_{t-1} + \rho X_t \\ \sigma_t^2 &= (1 - \rho)\sigma_{t-1}^2 + \rho(X_t - \mu_t)^T(X_t - \mu_t)\end{aligned}$$

Where ρ is the 2nd learning parameter, which is equivalent to α in re-implementation.

Finally, the Gaussians are sorted by $\omega_{i,t} \times \sigma_{i,t}^{-2}$ as the ideal background is static with minimal variance. After sorting, the first B out of the total K models are chosen as background pixels, and the rest are classified as objects.

Reproducible Component: Method and Result

This section presents a re-implementation of the Gaussian distributions of the adaptive mixture model for background subtraction proposed by Stauffer and Grimson. Aside from the main configured parameters used in the paper, the morphology was introduced to further improve the performance. Computing Gaussians directly is too expensive, so instead, an approximation is used in which the L2 distance between the mean value and the pixel colour is less than $2.5^2 * \text{variance}$. If the pixel is a part of the background, the mean, weight, and variance are updated accordingly. Otherwise, only the weight is updated via the decay equation. Additionally, only one learning parameter, α , was used.

Initial testing with Python yielded only 2 FPS at 480p, whereas the OpenCV model processed at 200-300 FPS at the same resolution. This was due to the limitations of Python as an interpreted language requiring dictionary calls when accessing properties in OOP patterns. To address this, the Mixture Gaussian Background Subtractor was converted to a Cython file, a C-compiled version of Python. Along with changing from object-oriented to array-based vectorized operations to take advantage of Hardware Acceleration compilation flags. This restructuring is to take advantage of SIMD, resulting in an increase in frame rate to 20 to 40 FPS. However, there is still a large gap between OpenCV and this implementation due to better vectorization, better Hardware Acceleration, and GPU usage.



Figure X: Stauffer and Grimson Gaussian Mixture Model Background Subtraction Result

Default parameters used in this reproducible test include: Number of Gaussian Kernels (K), Threshold (T), learning rate (α), and Morphology size (M). First, the timing benchmark was performed across different videos settings to evaluate the computational complexity as the single-parameter sweeps changed [2][3][4][5][6].

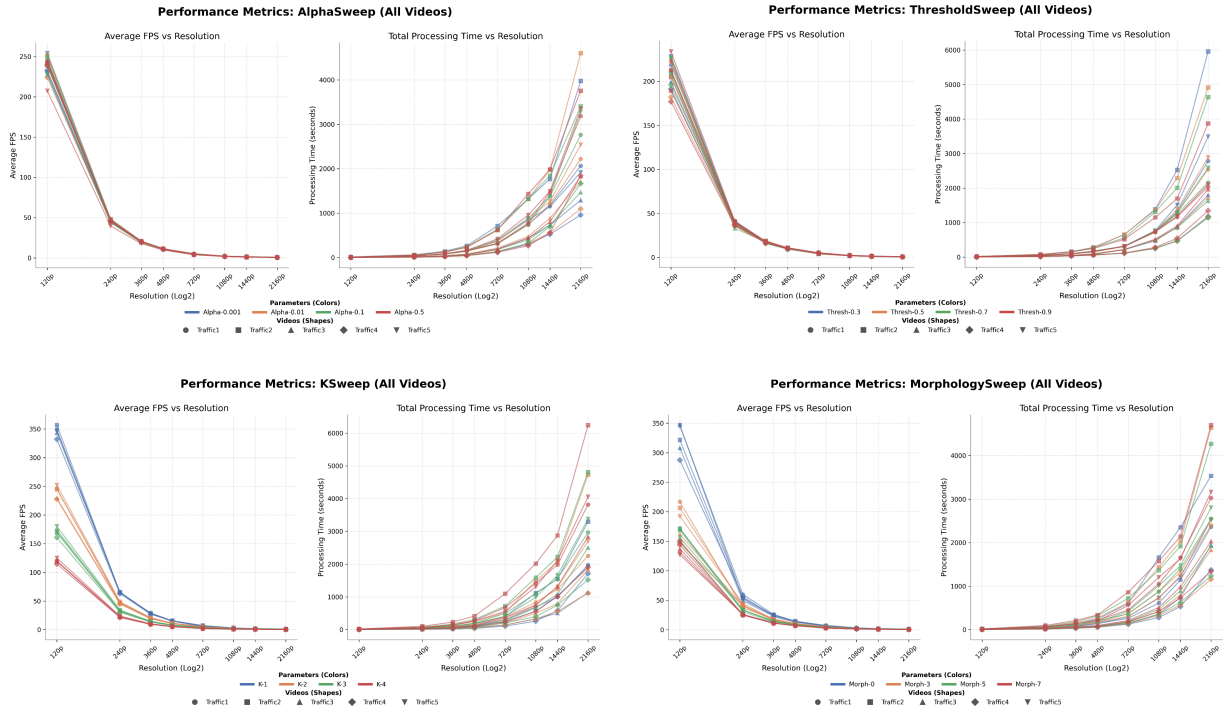


Figure x: Parameters Sweep Across all Videos: Learning Rate (Top-Left), Threshold (Top-Right), Gaussian Kernel (Bottom-Left), Morphology Size (Bottom-Right)

The most clear impact on processing speed is with the number of kernels and the morphology size. The processing time slightly improves with a higher learning rate, seen from the increase in FPS from $\alpha = 0.001$ to 0.5 at 120p, there is no clear trend. Similarly, there is no impact on speed with the threshold. From this, it is optimal to minimize the kernel and morphology size.

Moreover, the benchmark effectiveness was evaluated by comparing the re-implementation against Otsu and OpenCV MOG, which were selected as gold standards. The optimal settings are determined by performing a parameter sweep across K, T, and α on the re-implementation, before extracting the specified number of equally spaced frames from each video to compare against the OpenCV and Otsu results using SSIM to score their accuracy. The Average, Median and Variance of the SSIM scores were computed across the number of frames compared, which is then visualized using heatmaps to help highlight which parameter matched closely with the gold standards.

Below is an example of a benchmark comparison result from evaluating the “Traffic 2” video. It is visible that from the parameter curve plots, the Mean and Median scores improved with larger frame counts, while the variance showed the opposite trend. For the Heatmap average SSIM result, the output quality seems to improve with increasing value of all the configured parameters, but Otsu still has an overall higher quality output compared to OpenCV.

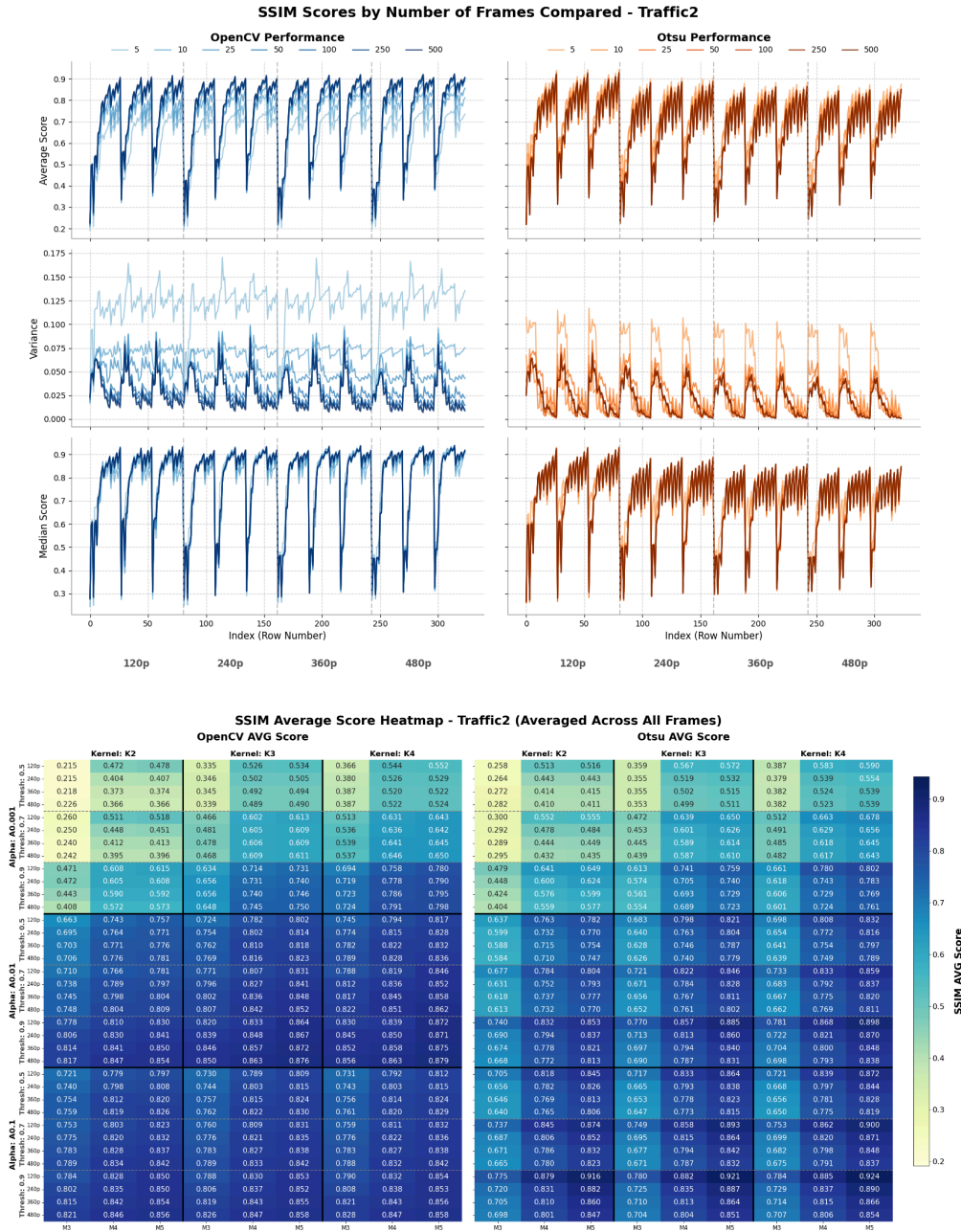


Figure x: Traffic2 SSIM Score by the number of frames (Top), SSIM Average Score Heatmap (Bottom)

Novelty component: extensions and analysis

The motivation behind using image segmentation, specifically with this method, was to implement it with security cameras. Storing HD footage is a challenging task: A single 1080p camera can use 24-48GB per day of recording [7]. Common encoding techniques such as H264 and H265 introduce major spatial and temporal redundancy. However, for an outdoor setting, periodic temporal data, such as leaves moving or water rippling, is not necessary, as the most significant actions happen at the macro scale.

This is where Stauffer and Grimson's method can be most impactful, given some additions: Along with Stauffer and Grimson's method, we add a Dilation operation repeated twice, Object Tracking,

and frame refreshing through a Refresh Index. The Dilation expands the cleaned objects that were created from previous Morphological Smoothing in the Background Subtractor implementation, which helps to reduce artifacts from the initial masks, such as trailing, which can happen due to objects moving fast, or light reflections, such as a moving headlight reflecting on a distant object in the background. Next, a Cython-based object detection module identifies objects of common interest: people, cars, pets, etc. If there is a significant motion in the frame and the object tracker classifies an object of interest, the portion of the video is passed along to be stored. To preserve some of the background information, the whole image was sampled every n frames, called the refresh index.

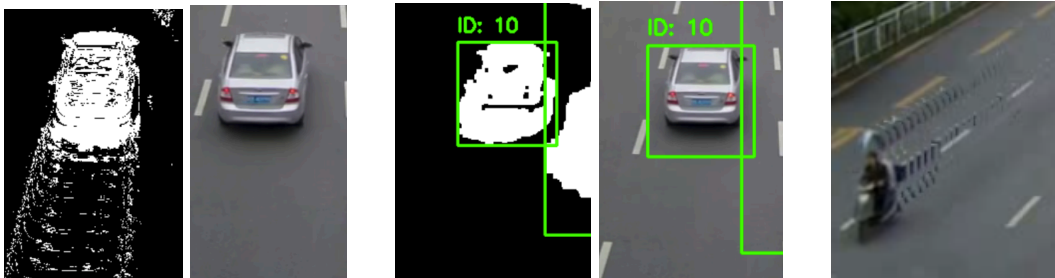


Figure x. Trailing artifact from a car (Left), and after Morphology Smoothing and Object detection (Center), and trailing compression artifacting (Right)

To optimize for data compression, the best parameters identified in the previous section were used as a baseline, and the newly added features were evaluated on top of them. Two features were tested: Object Detection and the Refresh Index. When Object Detection was enabled, only regions of the frame containing detected objects that overlapped with the foreground mask were saved. The Refresh Index was swept across values of 10, 20, and 30 frames, determining how often a full background frame was captured to preserve scene context.

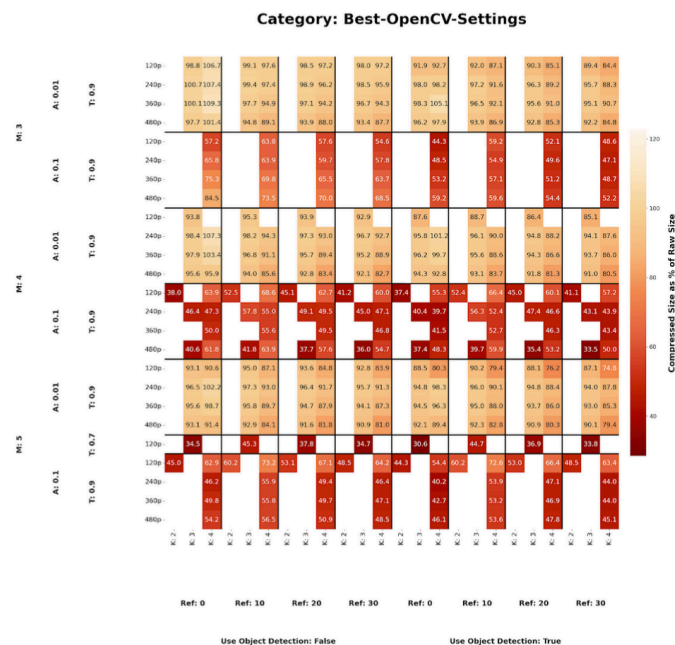


Fig x. Heatmap of Data Compression

The heatmap showed that increasing the learning rate had by far the most impact on data compression, with $\alpha = 0.1$ reducing the data by half. A higher morphology size and lower kernel

number also had an impact. A higher morphology smoothens the image spatially, which reduces the data. A lower kernel number may be less susceptible to noise, which increases compression. When there are fewer kernels, only the highest weight (lowest variance) kernels survive, so picking a proper threshold number will clearly separate the background and foreground with more certainty. Additionally, Object detection had <1% improvement in compression, and the refresh window had no clear trend for compression.

Discussions and Conclusion

The method proposed by Stauffer and Grimson, Gaussian Mixture Model (GMM) background subtraction, seems to correctly separate moving vehicles from the static background. However, when comparing a benchmark efficiently with OpenCV and Otsu, there is a significant performance gap even after the Cython compilation. The benchmark test confirmed that the Gaussian Kernel (K) and the Morphology kernel size (M) have the most effect on the computational speed, while Threshold (T) and the Learning Rate (α) have no clear impact on improving FPS. However, on the SSIM benchmark, learning rate and K seem to be the most influential parameters for the output quality; lowering both α and K degrades overall performance.

The novelty extension demonstrates that the Stauffer and Grimson method can serve as an effective preprocessing stage for intelligent video compression in security camera applications. The parameter sweep revealed that the learning rate had the most substantial impact on compression, with $\alpha = 0.1$ achieving roughly a 50% reduction in data. A higher learning rate causes the background model to absorb transient disturbances more quickly, resulting in sparser foreground masks. Morphology size and kernel count also contributed meaningfully, while object detection offered less than 1% compression improvement, likely because the foreground mask is already selective enough after morphological processing that most spurious detections are suppressed before object classification runs. The trailing artifact observed with fast-moving objects highlights a remaining limitation, though the double dilation step partially mitigates this by merging ghost regions into the detected bounding area.

In summary, this work successfully reproduces the Stauffer and Grimson GMM background subtraction method and evaluates it against OpenCV MOG and Otsu thresholding as benchmarks. While the Cython-based implementation significantly improved performance over a naive Python approach, a notable speed gap relative to OpenCV remains due to differences in vectorization and hardware acceleration. The SSIM benchmark confirmed that learning rate and kernel count are the most influential parameters for both output quality and computational speed. The novelty extension further demonstrated the method's practical utility in a security camera compression pipeline, where tuning the learning rate alone yielded substantial storage savings. Future work could explore GPU acceleration to close the performance gap and enable real-time deployment at higher resolutions.

References

- [1] C. Stauffer and W. E. L. Grimson, "Adaptive background mixture models for real-time tracking," Proceedings. 1999 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (Cat. No PR00149), Fort Collins, CO, USA, 1999, pp. 246-252 Vol. 2, doi: 10.1109/CVPR.1999.784637.
- [2] K. Majek, "4K Road traffic video for object detection and tracking - free download now!," *YouTube*, Jul. 16, 2018. [Online]. Available: <https://www.youtube.com/watch?v=MNn9qKG2UFI>
- [3] Editor, "Traffic on bridge at night," *YouTube*, May 28, 2011. [Online]. Available: <https://www.youtube.com/watch?v=sSfgai4YLIIs>
- [4] Sysvideo, "4K 8 Megapixel IP Camera Demo traffic car," *YouTube*, 2016. [Online]. Available: <https://www.youtube.com/watch?v=eO19UTm93GQ>
- [5] Pexels, "High angle footage of the city traffic," *Pexels*, 2020. [Online]. Available: <https://www.pexels.com/video/high-angle-footage-of-the-city-traffic-3696011/>
- [6] RDNE Stock project, "Drone footage of cars on a stop light," *Pexels*, 2021. [Online]. Available: <https://www.pexels.com/video/drone-footage-of-cars-on-a-stop-light-8783026/>
- [7] Safemo Pte. Ltd., "How much data does a home security camera use?," *Safemo*, Nov. 22, 2024. [Online]. Available: <https://safemo.com/blogs/home-security/how-much-data-does-a-security-camera-use>